Internship Handover Document

1. Project Overview

Project Name: Kashikart
Type: Desktop-based Tender Automation System (.exe Application)

Description

Kashikart is a Tender Data Automation & Processing System designed to automate the ingestion, processing, keyword filtering, and exposure of structured tender data. The system extracts tender information, processes it through a keyword engine, stores structured records in a centralized database, and provides dashboard access for monitoring and reporting.

The application is distributed as a Windows executable (.exe) using Electron, which integrates the React frontend and FastAPI backend.

Objective

- Automate tender data ingestion and processing
- Provide role-based secure access
- Implement keyword-based filtering (Signal > Noise model)
- Enable scheduled background jobs
- Provide export and Power BI integration support

2. System Architecture

The system follows a layered architecture pattern:

Client (Electron Desktop App)
↓
React.js Frontend
↓
FastAPI Routers (API Layer)
↓
Business Logic Layer (Services)
↓
ORM Layer (SQLAlchemy)
↓
MySQL Database

Additionally, scheduled background jobs:

APScheduler
↓
Scraping / Excel Processor
↓
Database Insert / Update

↓
Notification Engine

2.1. Architectural Highlights

Clear separation of concerns (Router → Service → ORM → DB)

Async database engine for API requests

Sync database engine for background jobs

Modular scraper design (extendable)

Role-based access control (RBAC)

Secure credential storage with encryption

Electron packaging for Windows executable distribution

3. Technology Stack

Backend

- FastAPI
- SQLAlchemy (Async + Sync)
- APScheduler
- JWT Authentication
- Alembic (Database Migrations)

Frontend

- React.js
- Dashboard UI for Tender Monitoring

Desktop Packaging

- Electron (Bundled React + FastAPI into .exe)

Database

- MySQL (Production)
- SQLite (Local fallback mode)

Integrations

- Excel Export
- Power BI Connector

4. Folder Structure Overview

Root Structure

tender-automation-system/

- backend/ (FastAPI Core)
- frontend/ (React Dashboard)
- database/ (Schema & migrations)
- powerbi/ (Power BI configs)
- docs/
- README.md

5. Backend Architecture (Core System)

Entry Point

- main.py → FastAPI application initialization

Core Layer (Infrastructure)

Responsible for:

- Environment configuration
- Database connection
- Security configuration
- Scheduler setup

Key Files:

- config.py
- database.py
- security.py
- scheduler.py

Authentication Module

Handles:

- User login
- Role-based authorization
- JWT token management

Routes:

- /login
- /users

Implements RBAC via dependency injection.

ORM Models

Database entities:

- Tender
- Keyword
- Source
- FetchLog
- Notification

These represent structured business objects stored in MySQL.

Routers (API Layer)

Responsible for:

- CRUD operations
- Manual "Fetch Now"
- Notifications management
- Source management

Each router calls its respective service layer.

Services Layer (Business Logic – Critical Layer)

This is the core processing engine.

Handles:

- Tender processing
- Keyword matching
- Change detection
- Notification triggering
- Data filtering

This layer ensures:

- Business rules are centralized
- Clean separation from API layer

Keyword Engine (Signal > Noise Layer)

Responsible for:

- Case-insensitive matching
- Priority scoring
- Document parsing
- Filtering irrelevant tenders

Scraping Engine (Extendable Module)

Modular scraper structure:

- Base scraper
- Login-based scraper
- HTML scraper
- PDF scraper
- Portal scraper

Note:
Currently not active for future expansion. Can be re-enabled if scraping-based ingestion is required.

Scheduler System

Uses APScheduler to:

- Run hourly jobs
- Run daily ingestion
- Trigger automated data processing
- Execute notification checks

Jobs are defined in:

- scheduler/jobs.py
- scheduler/triggers.py

Notification System

Supports:

- Email notifications
- Desktop alerts
- User-specific preferences

Triggered after keyword match or data update.

6. Electron Packaging (.exe Build)

The system is packaged as a Windows desktop application:

Electron Shell
→ Loads React Frontend
→ Starts FastAPI backend
→ Connects to MySQL
→ Creates distributable .exe in /dist

Build Output:

- dist/ folder generated after build
- Backend runs as local server inside packaged app

Purpose:

- Allow non-technical users to run system without manual backend setup

7. Team Structure & Roles

Role	Responsibility
Backend Developer	**API design, business logic, DB integration**
Frontend Developer	**Dashboard UI, API integration**
Database Engineer	**Schema design, migrations**
QA	**Testing & validation**
Excel & Power BI	**Web Scarping Using Excel and Creating Dashboard Using Power BI**

- **Backend Developer:** Abhishek Kolekar, Aditya Kumar
- **Frontend Developer:** Gaurav , Yashika Bawra, Sameer
- **Database Engineer:** Abhishek Kolekar, Aditya Kumar
- **QA:** Yashika Bawra, Sameer
- **Excel & Power BI:** Rishabh Singh

8. Module Ownership & Responsibilities

- **Authentication Module**

  Owner: Backend
  Status: Completed

- **Tender Processing Module**

  Owner: Backend
  Status: Active

- **Keyword Engine**

  Owner: Backend
  Status: Active

- **Scraping Engine**

  Owner: Future Enhancement

- **Power BI Integration**

  Owner: Integration Team

9. Pending Work & Recommendations

- Improve scraper automation (if required)
- Add more unit tests
- Optimize DB indexing for large datasets

- Improve change detection logic
- Implement logging dashboard
- Add Docker-based deployment (optional improvement)

10. Deployment Guide

Development Mode

1. Install Python dependencies
2. Configure .env
3. Run FastAPI server
4. Start React frontend

Production (.exe)

1. Build React app
2. Package via Electron
3. Generate dist folder
4. Install executable on client machine

11. My Contributions

- Designed and structured the backend architecture using a layered approach (Router → Service → ORM → Database)
- Implemented core business logic for tender processing
- Integrated APScheduler background jobs
- Designed modular scraping structure
- Implemented notification system
- Packaged system into Electron-based .exe
- Structured project for scalability and maintainability

12. Design Philosophy

- Clean Architecture Principles
- Modular Design
- Extensibility-first approach
- Security-focused implementation
- Separation of concerns
- Scalable database modeling